Advances in Cognitive Systems X (20XX) 1–16 Submitted X/20XX; published X/20XX

Toward Universal, Dynamic Goal Monitoring for Non-Omniscient Cognitive Agents

Geoffrey G. Short SHORTGE@UMD.EDU
James A. Reggia REGGIA@UMD.EDU
Rodolphe J. Gentili RODOLPHE@UMD.EDU
Neuroscience and Cognitive Science, University of Maryland, College Park, MD 20742 USA

Darsana P. Josyula DARSANA@CS.UMD.EDU
Department of Computer Science, Bowie State University, Bowie, MD 20715 USA

Abstract

Goal monitoring has proven essential for ensuring that goal-driven autonomous (GDA) agents in dynamic and uncertain environments continue to pursue only relevant and achievable goals as the state evolves. However, we argue that existing goal monitoring techniques are limited by their "static" nature and implicit reliance on an omniscient agent. In particular, we identify and detail two specific challenges for static goal monitoring by non-omniscient agents: (1) their inability to revise monitored conditions that are prone to evolution due to their specification in relation to local entities, and (2) their inability to revise monitored conditions when they are discovered to have been incorrectly, incompletely, or otherwise inappropriately defined. This paper introduces a novel dynamic goal monitoring technique designed to address a special case of the first limitation - predictable obsolescence of locally-defined monitored conditions that is induced by an agent's actions - while also offering foundational insights for addressing the second challenge in future work. Inspired by principles from neuroscience, such as corollary discharge and dynamic gating, our method enables agents to proactively update goal monitors during intentional actions. Implemented and tested within the MIDCA framework, we demonstrate that our method allows a partially observable agent to avoid the inappropriate abandonment of goals that remain feasible in the altered, post-action world, overcoming a key limitation of static goal monitors. This represents a significant advancement toward universal goal monitoring.

1. Motivation for Goal Monitoring and Current Limitations

The development of goal monitoring has been a significant advancement, equipping autonomous agents with an ability to track the continued validity and relevance of their objectives in dynamic and evolving environments. This section examines the motivation behind goal monitoring and evaluates its current limitations. Subsection 1.1 highlights the importance of goal monitoring in ensuring that agents only pursue goals whose rationale remains relevant in the current, evolving environment. Subsection 1.2 highlights scenarios where traditional goal monitoring techniques face challenges, particularly due to their reliance on static assumptions about monitored conditions. By identifying the limitations of existing methods, we set the stage for introducing a more dynamic and adaptive goal monitoring technique in subsequent sections.

© 20XX Cognitive Systems Foundation. All rights reserved.

1.1 Motivation for Goal Monitoring

Goal monitoring is an essential capability for GDA agents operating in dynamic and uncertain environments. In such environments, the state can shift unpredictably, potentially rendering goals irrelevant, obsolete, or unattainable. Without effective monitoring mechanisms, agents risk wasting effort on objectives misaligned with the current state of the world and/or their own current capacities. This challenge becomes especially pronounced for long-term goals, as the likelihood of deviations from initial conditions increases with time.

Before the advent of goal monitors (Cox & Dannenhauer, 2017; Dannenhauer & Cox, 2018; Dannenhauer, 2019), traditional agent frameworks relied on reactive, recovery-based strategies to avoid getting stuck in the pursuit of goals that had lost their relevance or feasibility (Cushing & Kambhampati, 2005; Georgeff & Lansky, 1987; Mendoza et al., 2015; Wilkins, 1985; Yoon & Givan, 2007). Agents addressed changes only after a plan failed or when they realized that their current goal had already been unexpectedly achieved due to independent evolution of the state. Common remedies included repairing the plan, formulating a new plan, or selecting a new goal. These approaches relied on evaluation of the status of the goal *itself*, limiting agents to reactive behaviors.

The innovation of goal monitoring lies in shifting the focus upstream—tracking the conditions that originally justified the generation and selection of a goal (i.e. its *rationale*). By associating monitors with these underlying justifications, cognitive agents gain the ability to detect when a goal's rationale no longer holds. This capability allows agents to abandon or modify goals proactively, rather than having to wait for goal/plan failures to trigger corrective action.

1.2 Current Limitations

1.2.1 "Static" Goal Monitoring and Reliance on an Omniscient Agent

Despite the initial successes of goal monitoring, its static nature limits its broader applicability. We characterize existing goal monitoring techniques as static because the monitored conditions remain fixed throughout the monitor's lifecycle. This is evident in the block diagram in Figure 1, which illustrates the integration of goal monitoring into the MIDCA cognitive architecture (Cox et al., 2016).

Two arrows point into the "Goal Monitors" box in the diagram. One represents the one-time generation of a goal monitor and is no longer relevant after that event. The other represents a recurring input, relayed each cycle, that provides information about changes in the world state. These updates are used solely to evaluate the *status* of the monitored conditions (i.e. whether they are satisfied) and determine whether the monitor should fire, as indicated by the output arrow from "Goal Monitors" to "Goals."

Critically, static goal monitoring does not involve an evaluation of whether the current *specification* of the monitored conditions *themselves* remain appropriate in light of changes to the world state or the agent's internal knowledge state. Thus, this approach implicitly assumes that the monitored conditions, as initially defined, are *indefeasible*. As a result, static goal monitoring is effectively limited to scenarios involving an omniscient agent—one that (1) knows all and can thus perfectly specify all relevant monitored conditions during initialization and (2) sees all and can thus comprehensively and accurately evaluate their status at all times.

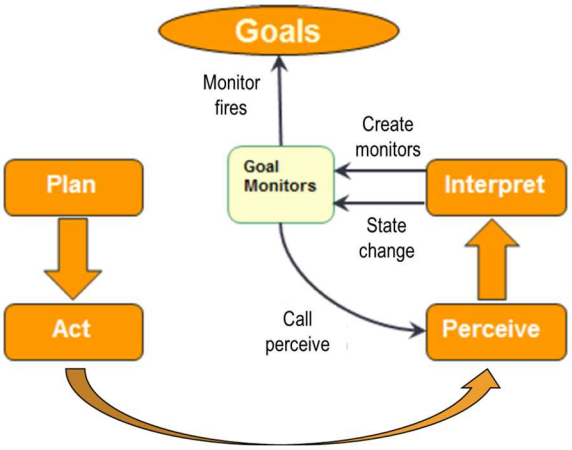


Figure 1. Conceptual illustration of original goal monitors in MIDCA. (Adapted from Dannenhauer & Cox, 2018; modified to include arrow from Act to Perceive)

In real-world GDA scenarios, agents are rarely omniscient and must compensate for partial observability or incomplete knowledge. These compensatory strategies introduce unique challenges for static monitors. Sections 1.2.2 and 1.2.3 introduce two common limitations of static goal monitoring in non-omniscient agents. First, partially observable agents often rely on proxy monitored conditions tied to local entities, since they cannot directly monitor globally defined ones. These proxy conditions are prone to evolution that static monitors cannot accommodate. Second, agents with incomplete knowledge may define monitored conditions based on assumptions that later require revision as understanding improves. However, static monitors cannot update their conditions once established, making them unsuitable for such cases.

1.2.2 Limiting Case 1: Partially Observable Agents and Locally-defined, Evolving Monitored Conditions

Compared to omniscient agents, partially observable agents can effectively evaluate the status of a more limited set of monitored conditions. Specifically, they cannot effectively or efficiently track conditions requiring global awareness of the world state. When such global monitored conditions are required, partially observable agents compensate by establishing proxy monitored conditions that are defined in relation to local entities that the agent can observe and reliably track. While these proxies provide a practical alternative, they are inherently subject to evolution in ways that static monitors cannot accommodate.

As a simple illustrative example, consider the blocks world domain (Winston, 1970) with an initial state, s_0, depicted in Figure 2. An agent with the goal *Goal(on(R, on(G,B))* aims to achieve a state where the red block (*R*) is stacked on top of the green block (*G*), which is stacked on the blue block (*B*). Upon formulating the goal, the agent establishes a goal monitor with monitored conditions corresponding to the goal's preconditions. The most fundamental preconditions, which most straightforwardly follow from the definition of the goal itself, relate to the *presence* of the entities in the goal (in this case, blocks *R*, *G,* and *B*). An omniscient agent could rely on more ideal,

globally-defined conditions such as *exists*(*R*), *exists*(*B*), and *exists*(*G*), or equivalently *in-world*(*R*), *in-world*(*B*), and *in-world*(*G*), as it can perfectly track the existence and location of each block.

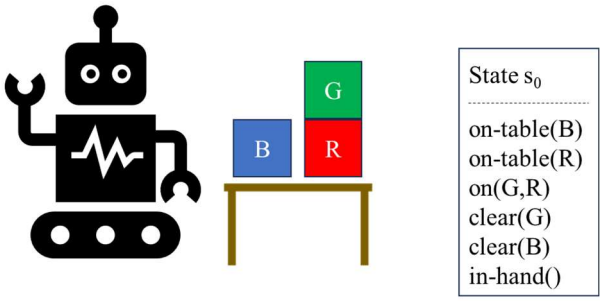


Figure 2. Initial state, s_0, of a simple scenario in the blocks world domain used to demonstrate where static goal monitoring cannot be successfully applied.

For a partially observable agent, however, global conditions like *exists*(*B*) are impractical to monitor. If block *B* is unknowingly moved to a location outside of the agent's view (e.g. inside a nearby box), the agent would perceive it as absent, even though the block still exists in the world. In such a case, it may be reasonable for the partially observable agent to abandon the goal, but this would not straightforwardly follow from globally defined monitored conditions like *exists*(*B*).

To address this limitation, partially observable agents compensate by establishing locally-defined monitored conditions. For example, the agent might assume that a block will remain at its current, observable location and define conditions like *on-table*(*R*), *on-table*(*B*), and *on(G,R)* as proxies for the more ideal, globally-defined conditions. While this approach ensures that monitored conditions are trackable, these proxies are tied to local variables and are subject to evolution, particularly as the agent acts on the environment. Static monitors, which lack the ability to adapt to evolving conditions, cannot accommodate this evolution.

A critical limitation of static goal monitoring emerges when a partially observable agent's intentional actions, performed as part of a plan to achieve its current goal, render one or more proxy monitored conditions obsolete. These actions, while predictably advancing the state of the proxy monitor conditions, can still trigger a static monitor to fire and the goal to be abandoned, even when the goal remains achievable. Continuing with the blocks world example, achieving the goal *Goal(on(R, on(G,B))* from the initial state s_0 requires actions that invalidate one or more proxy preconditions. A reasonable first step might involve moving *G* off of *R* and onto *B*, placing *G* where it ultimately needs to be while freeing *R* for its subsequent movement onto *G*. However, this action would invalidate the proxy monitored condition *on(G,R)*, causing the static monitor to fire and the goal to be abandoned. Yet, this abandonment would clearly be inappropriate. The action to remove *G* from *R*, and the associated outcome ¬*on(G,R)*, is an intentional and necessary step in the plan to achieve the current goal.

This example highlights a critical limitation of static goal monitoring: it cannot accommodate situations where the very actions required to achieve a goal result in a conflict with the proxy monitored preconditions of the associated goal monitor. Without the ability to update or revise the

definition of monitored conditions dynamically, the system risks prematurely abandoning goals that remain valid and achievable.

It is worth noting that modern planning frameworks that tightly integrate planning and acting (e.g., Castillo et al., 2012; Ghallab et al., 2016) are often designed to address related challenges—particularly the need to adapt to changes in the world that occur during execution. These systems typically support dynamic replanning when action outcomes deviate from expectations or when the preconditions for future actions are no longer satisfied. However, such systems focus on managing the viability of plans and actions, rather than tracking the ongoing validity of a goal's underlying rationale. In contrast, goal monitoring—whether static, as in current approaches, or dynamic, as developed in this work—emphasizes *goal-level metacognition*, wherein agents explicitly represent and continually evaluate the justifications for their goals over time. This capability is especially valuable in contexts involving long-term goals, partial observability, or delayed consequences, where execution-level signals may not provide timely or sufficient information. By enabling agents to revise goals based on changing justifications rather than only plan failures, goal monitoring provides a proactive complement to the reactive strengths of integrated planning systems.

1.2.3 Limiting Case 2: Agents with Imperfect or Incomplete Knowledge of Goal Preconditions

The second scenario where static goal monitoring proves insufficient arises when an agent operates with uncertain or incomplete beliefs about a goal's preconditions. While some goal preconditions, like those introduced in Section 1.2.2, are fundamental and certain (e.g., directly inferred from the goal specification), others are defeasible because they are based on assumptions, estimations, or reasoning processes that may later prove inaccurate.

To illustrate, consider again the blocks world domain and the preconditions such as *exists*(*R*), *exists*(*B*), and *exists*(*R*), or their proxy equivalents *on-table(B), on-table(R), and on(G,R)*. These preconditions are indisputably necessary because they directly follow from the definition of the goal *Goal(on(R, on(G,B))*. However, not all preconditions are as self-evident or straightforward. For example, an agent might reasonably conceive of a precondition requiring sufficient strength to grip or lift the blocks—one that does not follow directly from the goal specification. Identifying and specifying such preconditions often involves estimation (e.g., defining "sufficient strength") or assumptions, which can be flawed due to incomplete knowledge or lack of experience.

The fallibility of such assumptions poses challenges for static goal monitoring. Static monitors depend entirely on the monitored conditions specified at their inception, meaning any inaccuracies, omissions, or irrelevance in those monitors conditions persist throughout the monitor's lifecycle. As the agent interacts with its environment, observes changes, gathers data, and acquires new and better knowledge, it may discover that its initial assumptions require revision. Adjustments could involve modifying a monitored condition, removing one entirely, or adding new ones to better align with the actual requirements of the goal. However, static goal monitors lack the flexibility to accommodate such updates, leading to potential errors in goal management.

For example, consider a goal that depends on resource availability. These resources might include time (as in deadline-driven goals), space (as in navigation-based goals), materials or components (as in construction- or assembly-based goals), or even the agent's internal resources such as energy, knowledge, or confidence. Using deadline-driven goal reasoning (e.g., Kinny & George, 1991; Nirkhe et al., 1997) as the specific illustrative case, an agent might initially assume

that completing a task requires more time than is actually necessary. Based on this assumption, the agent creates a goal monitor tied to the remaining time before the deadline. During the execution of the task, the agent could acquire updated knowledge indicating that the task requires less time than originally estimated. Without the ability to revise its monitored conditions dynamically, the static monitor would trigger goal abandonment upon reaching the assumed time limit, despite the goal still being feasible within the actual time constraints.

This limitation underscores the importance of a dynamic goal monitoring process that can accommodate evolving beliefs about preconditions. Such systems would enable agents to refine their understanding of goal requirements in response to new evidence, avoiding unnecessary abandonment of achievable goals.

2. Method for Dynamic Goal Monitoring

As established in Section 1.2, there is a critical need to develop dynamic goal monitors that are more broadly applicable across diverse GDA scenarios. Two specific scenarios were introduced that exemplified this need:

1. Cases where *proxy monitored condition obsolescence is intentionally induced* by a partially observable agent's actions as it pursues its current goal.
2. Cases involving *defeasible precondition beliefs*, where a non-omniscient agent's assumptions about goal preconditions may be revised due to newly acquired knowledge.

These scenarios require distinct approaches.

In the first scenario, the agent intentionally modifies the world state through its actions. Consequently, the agent has privileged *access to foreknowledge* of the post-action world state. This foreknowledge empowers the agent to *proactively* update the definition of monitored conditions, ensuring that they will be appropriate in the anticipated world state.

In contrast, the second scenario requires more of a *reactive* approach to monitor revision. Here, even when the agent might recognize its precondition beliefs as defeasible, it still lacks the foreknowledge of when or how its knowledge state will be updated. Instead, the agent must be capable of detecting changes to its knowledge state that are relevant to its goal monitors and to be able to update them accordingly.

The detailed process for the first scenario is described in Section 2.1, with its neuroscientific and cognitive inspiration discussed in Appendix A. Appendix B discusses the second scenario and key considerations for its future development.

2.1 Proactive Goal Monitor Updates During Intentional Action

Our approach to proactively update goal monitors during intentional action is illustrated in Figure 3, with pseudocode detailing its core procedures provided in Algorithms 1 and 2. Since it builds upon the state-of-the-art goal monitoring framework, it has been developed in context of the same cognitive architecture, MIDCA. It assumes that goal monitors have been previously established and focuses solely on the process of updating these monitors. It also assumes that each goal in the agent's agenda is assigned a single monitor, which tracks a comprehensive set of conditions, including all preconditions of the monitored goal. Unlike prior work, which used multiple monitors per goal—each tied to individual preconditions—this method consolidates monitoring into a single

monitor per goal for simplicity. Finally, it assumes that goal monitors operate in parallel with the MIDCA cognitive cycle and "ping" at regular intervals, slightly shorter than the time required to process one cycle. At each ping, monitors verify that their conditions remain valid (i.e., that they remain present in the world state).

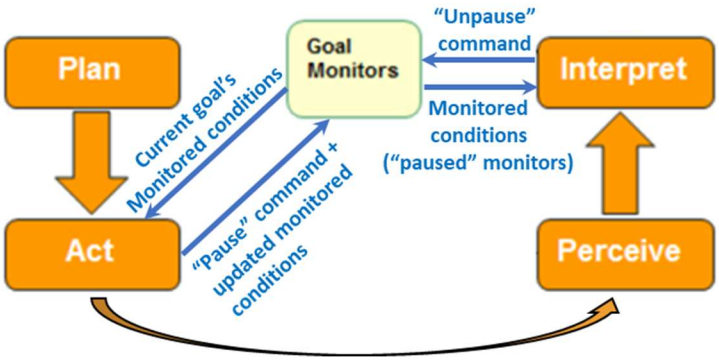


Figure 3. Conceptual illustration of dynamic goal monitoring (proactively updating goal monitors during intentional action). When Act determines that a monitored condition of the current goal will be affected by the selected action, it issues a command to pause the monitor and updates its monitored conditions. For any paused monitors, Interpret confirms the presence of the monitored conditions in the post-action world state and issues an "unpause" command.

Our method begins during the MIDCA Act phase, immediately after the agent selects an action intended to make progress toward its current goal. First, Act searches for the goal monitor associated with the current goal and retrieves its monitored conditions (arrow from "Goal Monitors" to "Act" in Figure 3; lines 2-4 in Algorithm 1). It is important to emphasize that the monitor associated with the current goal is the *only* monitor that is appropriate to update here because it is the only one that the agent considered when forming the plan and selecting the action. In other words, it is the only goal for which the plan or action is intentionally designed to make progress. In context of all other goals, the action is selected without consideration of the potential impacts and so its execution may result in a world state that unintentionally renders one or more of them infeasible. In such cases, it would be appropriate for the associated monitor to fire and for that goal to be abandoned. By contrast, for the goal being intentionally pursued, the agent must preemptively account for anticipated changes to avoid inappropriate monitor firing.

Next, the predicted results of the action and post-action world state are determined. For simpler, deterministic scenarios, this step is straightforward, as the action operator specifies the outcomes. These include positive results, which are atoms expected to be added to the world state, and negative results, which are atoms expected to be removed. For simplicity, this work assumes such prespecified outcomes. In more complex scenarios, the agent would indeed need to predict the outcomes of its actions. In such cases, the agent would likely rely on an internal forward model—a common mechanism in cognitive-motor control and robotic manipulation (Dearden & Demiris, 2005; Gentili et al., 2004; Gentili et al., 2015; Miall & Wolpert, 1996; Wolpert & Kawato, 1998)

Algorithm 1 Proactively update the current goal's monitor to account for the effects of the selected action. The algorithm assumes that an action, α_s, has just been selected and that monitors, goals, and actions use the operator style. A monitor, ϕ, contains a set of monitored conditions, *mcs*, a monitored goal, *g,* and a status (paused/unpaused). An action operator contains a name, preconditions, and effects, and an effect, *eff*, includes a predicate, subject, and object. **Input:** current goal g_c, selected action α_s, goal monitor set $\hat{\Gamma}$. **Output:** (updated) goal monitor set $\hat{\Gamma}$

```
1:  procedure update_G_monitors(g_c, α_s, Γ̂)
2:      for φ in Γ̂ do
3:          if φ.g = g_c then
4:              for p in φ.mcs do
5:                  if p in α_s.eff⁻ then
                        ▷ pause the monitor
6:                      φ.is-paused ← True
7:                      for pe in α_s.eff⁺ do
8:                          if pe.subject= p.subject then
                                ▷ revise condition that α will invalidate
9:                              replace p in φ.mcs with pe
10:     return Γ̂
```

The monitored conditions of the current goal's monitor are then compared against the negative results of the action (line 5 in Algorithm 1). If any monitored condition matches a negative result, it indicates that the action will render the condition false in the post-action world state. To prevent the monitor from firing due to this anticipated change, the affected monitored condition must be updated. Before updating the monitored conditions, the monitor is temporarily set to a "paused" status (line 6 in Algorithm 1). This precaution prevents inappropriate firing that would otherwise be caused by a temporary mismatch between the current, pre-action, world state and the monitored conditions that have been updated to reflect the anticipated post-action world state. Once paused, the action's positive results are searched for an atom with the same subject as the soon-to-be-outdated/negated monitored condition. The relevant positive result is then used to overwrite the monitored condition, reflecting the anticipated post-action state (lines 7-9 in Algorithm 1). The updates to the monitored conditions, along with the "pause" command, are transmitted to the goal monitor (arrow from "Act" to "Goal Monitor" in Figure 3).

At this point, the updated goal monitor remains paused until the action has been executed and the anticipated post-action world state is confirmed. The process of unpausing the monitor (Algorithm 2) occurs during the Interpret phase, after the agent has observed the post-action world state in the Perceive phase. First, Interpret identifies monitors with a 'paused' status and retrieves their monitored conditions (arrow from "Goal Monitors" to "Interpret" in Figure 3; lines 2-4 in Algorithm 2). For each such monitor, it checks the world state to confirm the presence of all updated monitored conditions. If the conditions are verified, the monitor's status is switched to "unpaused," restoring its active functionality (lines 4-5 in Algorithm 2). Otherwise, further evaluation is needed to determine appropriate next steps, such as retrying the action, allowing the monitor to fire, or

other strategies (lines 6-7 in Algorithm 2). While these considerations fall outside the scope of this work, they present promising directions for future research.

Algorithm 2 Unpause goal monitors. The algorithm assumes that the monitor uses the operator style and that a monitor, ϕ, contains a set of monitored conditions, *mcs*, a monitored goal, *g,* and a status (paused/unpaused). **Input:** goal monitor set $\hat{\Gamma}$, world state, *s.* **Output:** (updated) goal monitor set $\hat{\Gamma}$

```
1:  procedure unpause_G_monitors(Γ̂, s)
2:      for φ in Γ̂ do
3:          if φ.is-paused then
                ▷ confirm all conditions now hold in the world
4:              if all p in φ.mcs are in s then
                    ▷ unpause the monitor
5:                  φ.is-paused ← False
6:              else
                    ▷ further evaluation may be required
7:                  evaluate(φ.mcs,s)
8:      return Γ̂
```

3. Implementation and Results

We implemented our method for dynamic goal monitoring—specifically the proactive updating of proxy goal monitors that would otherwise become obsolete due to intentional actions— within the MIDCA framework (Cox et al., 2016) and demonstrated it in context of the logistics domain (Veloso, 1994). The combination of MIDCA and logistics domain was chosen to allow a direct comparison between the dynamic goal monitors developed here and the static goal monitors from prior work (Dannenhauer & Cox, 2018). By using the same cognitive architecture and domain, the study can better highlight the advantages of dynamic goal monitoring over the static baseline.

Section 3.1 provides an overview of the logistics domain, including minor modifications made for this demonstration. It describes the initial world state and the goal preconditions assumed during the experiments. Finally, the results of the demonstration are presented in Section 3.2, showcasing the behavior and outcomes for agents using static versus dynamic goal monitoring.

3.1 Modified Logistics Domain

The logistics domain involves delivering packages from their origin locations to specified destination locations. Deliveries are carried out using trucks and airplanes situated at various locations within the domain. The domain also includes cities, each of which contains multiple locations. The numerical IDs of cities and locations correspond in the following way: the first digit of the location ID corresponds to the city it is contained within. For example, *loc01* is in *city0*, and *loc12* is in *city1*.

At initialization, the agent is assumed to have complete knowledge of the packages available in warehouses and the locations of all planes, trucks, locations, and cities. However, this knowledge

is not assumed to remain complete throughout the simulation. As the environment evolves, the agent must rely on partial observability and ongoing perception to maintain and update its knowledge state. The agent receives orders and formulates them into delivery goals. The order *deliver(package01, loc12)* would thus be translated into the goal *Goal(package01, loc01, loc12, predicate: obj-at)*. Each delivery goal operator has preconditions, ensuring that the goal is only initiated under appropriate conditions.

As in the prior work, each delivery goal includes a precondition requiring the package to be at its initial location, specified as *obj-at(package01, loc01)*. This ensures that the goal is maintained in the agent's goal agenda only if the package remains where it is supposed to be (and hasn't been misplaced, stolen, or otherwise lost in the time since the goal was initially established). An additional precondition is introduced in this work to further demonstrate the versatility of dynamic goal monitoring. The precondition is specific to intercity[1] deliveries and specifies that an airplane must be available in the same city as the package's starting location. This prevents "costly" deliveries requiring a plane to be flown a long distance from a different city solely for the purpose of picking up the relevant package. This condition is specified as *airplane-at(plane??, loc#?)*, where "plane??" represents the idea that any plane is acceptable and "loc#?" stands for any location in *city#*. Together, the two preconditions for the intercity delivery goal *Goal(package01, loc01, loc12, predicate: obj-at)* are *obj-at(package01, loc01)* and *airplane-at(plane??, loc0?)*.

3.2 Results

To demonstrate the broader applicability of dynamic goal monitoring, two experimental runs of MIDCA were conducted in the modified logistics domain. The first run incorporated baseline static goal monitoring, while the other used the dynamic goal monitoring process developed in this work. This section summarizes the results, focusing on the most salient steps of the process.

Across both runs, the initial state (Figure 4) included an order to deliver *package01* from *loc01* to *loc12*. At *loc01*, a truck (*truck01*) was available, and an airplane (*plane01*) was located at *loc02*, in the same city (*city0*) as the package's origin. Since an order for a package, *deliver(package01, loc12),* existed and the package is present in the origin warehouse, *obj-at(package01, loc01)*, a delivery goal, *Goal(package01, loc01, loc12, predicate: obj-at)*, was generated during the MIDCA Interpret phase. This goal is an intercity delivery goal, since it originates in *loc01 (*in *city0)* and is delivered to *loc12* (in *city1*). At the time the goal is generated, a corresponding goal monitor was created. The monitor tracked the intercity delivery goal's two preconditions: *airplane-at(plane??, loc0?)* and *obj-at(package01, loc01)*. *Goal(package01, loc01, loc12, predicate: obj-at)* was selected as the current goal in the MIDCA Intend phase, and the JSHOP planner (Nau, 1999) then generated a plan (Figure 5) to achieve it as part of the Plan phase. The first action in the plan, *load-truck(package01, truck01, loc01)*, was selected. It was at this point that the behavior of the static and dynamic goal monitors diverged.

[1] Per the relationship between location ids and city ids, an intercity delivery goal is one where the first digit of the origin location id differs from that of the destination location id.

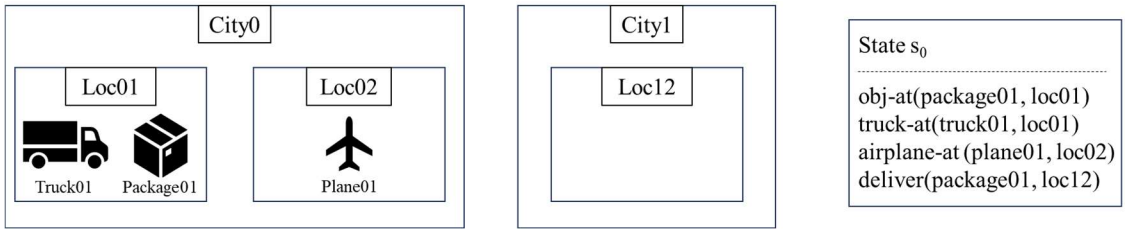


Figure 4. Initial state, s_0, of the modified logistics domain that was used to demonstrate our dynamic goal monitoring approach (proactively updating goal monitors during intentional action).

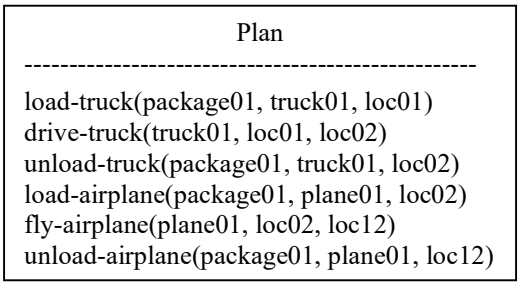


Figure 5. Plan generated by the JSHOP planner of MIDCA's Plan phase to achieve the *Goal(package01, loc01, loc12, predicate: obj-at)*.

3.2.1 Static Goal Monitoring Results

In the static goal monitoring run (Figure 6), the selected action was then executed. This updated the world state by adding an atom corresponding to the result *in-truck(package01, truck01)* and removing the atom *obj-at(package01, loc01)*. When the goal monitor subsequently ran its next check of the world state, it detected that one of its monitored conditions, *obj-at(package01, loc01)*, was no longer present. Consequently, the monitor fired and abandoned the goal. With no remaining goals, the run ended.

Cycle	End of Phase	Monitored conditions		Monitor status	Goal status
		mc_1	mc_2		
1	Interpret	*airplane-at(plane??, loc0?)*	*obj-at(package01, loc01)*	Unpaused	Active
	Act (load-truck)	↓	↓	↓	↓
2	Interpret			Fired	Abandoned
	Act	END OF RUN			

Figure 6. Results from the baseline experimental run of MIDCA in the modified logistics domain incorporating *static* goal monitoring.

3.2.2 Dynamic Goal Monitoring Results

In contrast, dynamic goal monitoring proved robust to action-induced world state changes. In the dynamic goal monitoring run (Figure 7), the current goal's monitored conditions, *airplane-at(plane??, loc0?)* and *obj-at(package01, loc01)*, were retrieved before executing the selected action. These were compared to the negative results specified in the selected actions' operator. The set of negative results contained one of the monitored conditions, *obj-at(package01, loc01)*, so the status of the goal monitor was set to *"paused"*. With the goal monitor paused, the selected action's positive results were next searched for an entity with the same subject, *package01*, as the soon-to-be-obsolete monitored condition. This identified the positive action result *in-truck(package01, truck01)*, which was used to overwrite the monitored condition. The action was then executed, and the world state was updated by adding an atom corresponding to the result *in-truck(package01, truck01)* and removing the atom *obj-at(package01, loc01)*.

Cycle	End of Phase	Monitored conditions		Monitor status	Goal status
		mc_1	mc_2		
1	Interpret	*airplane-at(plane??, loc0?)*	*obj-at(package01, loc01)*	Unpaused	Active
	Act (load-truck)		*in-truck(package01, truck01)*	Paused	
2	Interpret			Unpaused	
	Act (drive-truck)				
3	Interpret				
	Act (unload-truck)		*obj-at(package01, loc02)*	Paused	
4	Interpret			Unpaused	
	Act (load-airplane)		*in-plane(package01, plane01)*	Paused	
5	Interpret			Unpaused	
	Act (fly-airplane)	*airplane-at(plane??, loc1?)*		Paused	
6	Interpret			Unpaused	
	Act (unload-airplane)		*obj-at(package01, loc12)*	Paused	
7	Interpret			Unpaused	
	Evaluate			Terminated	Achieved

Figure 7. Results from the baseline experimental run of MIDCA in the modified logistics domain incorporating *dynamic* goal monitoring.

At this point, the goal monitor was paused and its updated monitored conditions were *airplane-at(plane??, loc0?)* and *in-truck(package01, truck01)*. The agent then perceived the post-action world state as part of the MIDCA Perceive phase, and the world state was passed to Interpret. During the Interpret phase, a search retrieved the monitored conditions of the paused goal monitor. A subsequent search through the world state confirmed the presence of both monitored conditions. After verifying the updated condition in the post-action world state, the monitor's status was set to "unpaused", allowing the agent to continue pursuing the goal.

This process was repeated for subsequent actions, such as *unload-truck(package01, truck01, loc02)* and *load-airplane(package01, plane01, loc02)*, whose results would negate one or more monitored conditions, with the monitored conditions being dynamically updated at each step. The

final goal monitor update occurred after *fly-airplane(plane01, loc02, loc12)*, with the condition *airplane-at(plane??, loc0?)* updated to *airplane-at(plane??, loc1?)*. The package was successfully delivered to *loc12* following the final action, *unload-airplane(package01, plane01, loc12)*.

4. Conclusion and Future Work

This work examined two key limitations of static goal monitoring: the obsolescence of locally defined preconditions caused by intentional actions, and the rigidity of monitors defined using defeasible precondition beliefs. While integrated planning and acting frameworks (e.g., Castillo et al., 2012; Ghallab et al., 2016) offer powerful tools for adapting to execution failures and changes in the environment, they do not maintain explicit representations of goal rationale or support introspective monitoring of that rationale over time. In contrast, goal monitoring—particularly in its dynamic form—adds a complementary, metacognitive layer that enables agents to evaluate and revise goals based on evolving internal justifications, not just execution outcomes. To address the limitations of both static goal monitoring and integrated planning frameworks, we introduced a dynamic goal monitoring approach that enables agents to proactively revise monitors in anticipation of state changes induced by their own actions. This method replaces static monitors in settings where monitored conditions are expected to evolve and complements integrated planning systems by adding a metacognitive layer that supports goal-level reasoning alongside action-level adaptation.

More specifically, we implemented this technique within the MIDCA framework and demonstrated that it prevents premature goal abandonment in partially observable environments, enabling more robust goal pursuit even when locally defined monitored conditions evolve. Appendix C confirms that the approach imposes minimal computational overhead—even when scaled to dozens of monitored conditions, well beyond typical use cases—reinforcing its practicality for real-time use. The demonstration served as an initial proof of concept, showcasing the benefits of dynamic goal monitoring in a simplified scenario—one involving a single goal achieved through a straightforward plan with immediate action execution. The resulting tradeoff space between goal execution without monitoring (baseline MIDCA), static goal monitoring, and dynamic goal monitoring in such scenarios is summarized in Table 1. We expect the benefits of our approach to be even more pronounced in complex scenarios, particularly those involving long-term goals with significant delays between actions. For example, an agent pursuing one goal might encounter an unavailable action and temporarily suspend that goal to focus on another. By ensuring that monitored conditions remain accurate and up to date while the goal is shelved, the agent can seamlessly resume pursuit of the original goal when appropriate, using the correct preconditions upon resumption.

This work also offers a preliminary computational instantiation of a broader theoretical claim (Feinberg, 1978; Subramanian, Alers, & Sommer, 2019): that mental simulation and internal predictive signals—such as corollary discharge, traditionally studied in motor control (e.g., Gentili et al., 2015; Mohan et al., 2010)—can be extended to higher-order cognitive processes. By evaluating how the predicted outcomes of intentional actions will affect goal-relevant justifications, dynamic goal monitoring enables agents to shift from reactive control toward anticipatory reasoning. In this sense, the contribution of this work is not only functional but also theoretical,

advancing the understanding of how introspective predictive mechanisms might generalize across levels within a cognitive–motor architecture.

These results mark a significant step toward more universally applicable goal monitoring. However, important challenges still remain—both in integrating our procedures more precisely within the dual-cycle cognitive architecture and in expanding their functionality to address broader classes of GDA scenarios.

Table 1. Summary of goal achievement behavior and incremental computational costs across three goal monitoring configurations (in MIDCA).

Monitoring Configuration	Goal Achievement Behavior	Incremental Computational Cost
No Goal Monitoring	• Achieves some goals, but may become stuck on goals that become unachievable due to external changes.	• None
Static Goal Monitoring	• Allows appropriate abandonment of goals when external events make them unachievable. • Fails to achieve (inappropriately abandons) goals requiring action-induced state changes.	• Adds computational cost of establishing monitors and checking monitored conditions against the world state each cycle.
Dynamic Goal Monitoring	• Still appropriately abandons goals in response to external events. • Achieves all feasible goals, including those requiring agent-driven state changes.	• Adds cost of proactive monitor updates (Algorithm 1) and unpausing (Algorithm 2). • Less than 1% additional overhead relative to static goal monitoring.

While designing the dynamic goal monitoring procedures (Algorithms 1 and 2), our primary focus was on demonstrating their functionality and ensuring that their effects occurred at the appropriate point in the reasoning cycle to support intended behavior. More nuanced considerations of architectural placement—namely, which components belong in MIDCA's cognitive versus metacognitive levels—were deferred in favor of simplicity and consistency with conventions established in prior MIDCA literature. For instance, Algorithm 2 was placed within the Interpret phase because its function parallels that of baseline MIDCA's Interpret module: comparing the perceived world state to an expected one. In Algorithm 2, however, the expected state is derived from updated monitored conditions—effectively a form of goal-specific working memory—rather than from MIDCA's central world model, as in standard Interpret functionality.

Nonetheless, several procedures currently located within Interpret—and in dynamic goal monitoring more broadly—such as those that update the goal or monitor set, are arguably more metacognitive in nature, as they involve modifying internal cognitive representations rather than processing information about the external world. The same is true of several operations in Algorithm 1, currently situated in the Act phase. From this perspective, such procedures may be more appropriately relocated to MIDCA's metacognitive layer. Future work will explore these distinctions more deliberately, refining the architectural placement of dynamic monitoring operations to better reflect MIDCA's layered reasoning structure. This reorganization will also raise

questions about timing and control flow across cognitive and metacognitive levels, which we anticipate will be a central focus of the integration effort.

As a complement to this structural work, separate future work will further extend the functionality and applicability of dynamic goal monitoring. In particular, we aim to implement and evaluate its use in scenarios that remain unaddressed—most notably, those involving defeasible precondition beliefs, as outlined in Appendix B. In doing so, we hope to advance toward a more organized and flexible foundation for goal management in autonomous agents.

References

Anderson, M. L., Gomaa, W., Grant, J., & Perlis, D. (2008). Active logic semantics for a single agent in a static world. *Artificial Intelligence, 172*(8-9), 1045-1063.

Castillo, L., Fdez-Olivares, J., & García-Pérez, Ó. (2012). PELEA: Planning, execution and learning architecture. In *Proceedings of the 22nd International Conference on Automated Planning and Scheduling (ICAPS), System Demonstrations Track*.

Cox, M. T., & Dannenhauer, Z. A. (2017). Perceptual goal monitors for cognitive agents in changing environments. In *Proceedings of the Fifth Annual Conference on Advances in Cognitive Systems, Poster Collection* (pp. 1-16).

Cox, M., Alavi, Z., Dannenhauer, D., Eyorokon, V., Munoz-Avila, H., & Perlis, D. (2016). MIDCA: A metacognitive, integrated dual-cycle architecture for self-regulated autonomy. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 30, No. 1).

Cushing, W., & Kambhampati, S. (2005). Replanning: A new perspective. *Proceedings of the International Conference on Automated Planning and Scheduling Monterey, USA*, (pp. 13-16).

Dannenhauer, Z. A. (2019). Anticipation in dynamic environments: Deciding what to monitor. (Doctoral dissertation, Wright State University)

Dannenhauer, Z. A., & Cox, M. T. (2018). Rationale-based perceptual monitors. *AI Communications, 31*(2), 197-212.

Dearden, A., & Demiris, Y. (2005). Learning forward models for robots. In *IJCAI* (pp. 1440-1445).

Elgot-Drapkin, J. J., & Perlis, D. (1990). Reasoning situated in time I: Basic concepts. *Journal of Experimental & Theoretical Artificial Intelligence, 2*(1), 75-98.

Feinberg, I. (1978). Efference copy and corollary discharge: implications for thinking and its disorders. *Schizophrenia bulletin*, *4*(4), 636.

Frank, M. J. (2005). When and when not to use your subthalamic nucleus: Lessons from a computational model of the basal ganglia. In *Modelling natural action selection: Proceedings of an international workshop* (pp. 53-60). AISB.

Gentili, R., Cahouet, V., Ballay, Y., & Papaxanthis, C. (2004). Inertial properties of the arm are accurately predicted during motor imagery. *Behavioural Brain Research, 155*(2), 231-239.

Gentili, R., Oh, H., Huang, D., Katz, G., Miller, R., & Reggia, J. (2015). A neural architecture for performing actual and mentally simulated movements during self-intended and observed bimanual arm reaching movements. *International Journal of Social Robotics*, *7*, 371-392.

Georgeff, M. P., & Lansky, A. L. (1987). Reactive reasoning and planning. In *Proceedings of the Sixth National Conference on Artificial Intelligence (AAAI-87)* (pp. 677–682). AAAI Press.

Ghallab, M., Nau, D., & Traverso, P. (2016). *Automated planning and acting*. Cambridge University Press.

Herron, A., & Josyula, D. P. (2023). An Active-Logic Based Agent's Reasoning for Avoiding Futile Action Repetition. *INTELLI 2023, 35*.

Johnson, M. K., & Hirst, W. (1993). MEM: Memory subsystems as processes. In *Theories of memory* (pp. 241-286). Psychology Press.

Josyula, D., & M'Bale, K. (2013). Bounded metacognition. In *Fifth international conference on advanced cognitive technologies and applications* (pp. 147-152).

Kinny, D., & George, M. (1991). Commitment and Effectiveness of Situated Agents. In *IJCAI-91* (pp. 82-88).

Miall, R. C., & Wolpert, D. M. (1996). Forward models for physiological motor control. *Neural Networks, 9*(8), 1265-1279.

Mendoza, J. P., Veloso, M., & Simmons, R. (2015). Plan execution monitoring through detection of unmet expectations about action outcomes. In *2015 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 3247–3252). IEEE.

Mohan, V., Morasso, P., Metta, G., & Kasderidis, S. (2010). Actions and imagined actions in cognitive robots. In *Perception-Action Cycle: Models, Architectures, and Hardware* (pp. 539-572). New York, NY: Springer New York.

Murray, E. A., Wise, S. P., & Graham, K. S. (2017). *The Evolution of Memory Systems: Ancestors, anatomy, and adaptations.* Oxford University Press.

Nau, D., Cao, Y., Lotem, A., & Muñoz-Avila, H. (1999). SHOP: Simple hierarchical ordered planner. In *Proceedings of the 16th International Joint Conference on Artificial Intelligence (IJCAI-99)* (pp. 968–973).

Nelson, T. O. (1990). Metamemory: A theoretical framework and new findings. In *Psychology of learning and motivation* (Vol. 26, pp. 125-173). Academic Press.

Nirkhe, M., Kraus, S., Miller, M., & Perlis, D. (1997). How to (plan to) meet a deadline between now and then. *Journal of Logic and Computation*, *7*(1), 109-156.

O'Reilly, R. C. (2006). Biologically based computational models of high-level cognition. *Science, 314*(5796), 91-94.

Sperry, R. W. (1950). Neural basis of the spontaneous optokinetic response produced by visual inversion. *Journal of comparative and physiological psychology*, *43*(6), 482.

Subramanian, D., Alers, A., & Sommer, M. A. (2019). Corollary discharge for action and cognition. *Biological Psychiatry: Cognitive Neuroscience and Neuroimaging, 4*(9), 782-790.

Veloso, M. M. (1994). Planning and learning by analogical reasoning. Berlin: Springer.

Wilkins, D. E. (1985). Recovering from execution errors in SIPE. *Computational Intelligence, 1*(1), 33–45.

Winston, P. H. (1970). Learning structural descriptions from examples. (Doctoral dissertation, Massachusetts Institute of Technology)

Wolpert, D. M., & Kawato, M. (1998). Multiple paired forward and inverse models for motor control. *Neural Networks, 11*(7-8), 1317-1329.

Yoon, S., Fern, A., & Givan, R. (2007). FF-replan: A baseline for probabilistic planning. In *Proceedings of the 17th International Conference on Automated Planning and Scheduling (ICAPS-07)* (pp. 352–359).

Appendix A: Neuroscientific and Cognitive Inspiration for Dynamic Goal Monitoring

Our approach to updating goal monitors during intentional action (Section 2.1) was motivated by conceptual insights from neuroscience and human cognition. This appendix highlights two relevant principles that parallel procedures in the proposed method: corollary discharge, central to cognitive motor control models, and dynamic gating processes in the cortico-basal ganglia-thalamocortical loop, essential for action initiation and working memory updates. These principles serve as high-level analogies that motivate aspects of our approach to dynamic goal monitor updates.

A crucial aspect of updating goal monitors is determining whether a selected action renders a monitored condition obsolete. This process involves predicting (or retrieving) the outcomes of the selected action and anticipating the resulting post-action (to-be-perceived) world state. Such predictive mechanisms have a strong neuroscientific parallel in the corollary discharge signal, a concept rooted in motor control research (Sperry, 1950; Miall & Wolpert, 1996).

Traditionally, corollary discharge has been understood solely in the context of motor control, where it contributes to real-time control and more stable, smooth execution of movement. However, it has been argued that its role may extend beyond motor functions, contributing to higher-level cognitive processes (Feinberg, 1978; Subramanian, Alers, & Sommer, 2019). This perspective aligns with the brain's evolutionary tendency for exaptation, where neural components initially evolved for more basic, low-level functions are later co-opted to more complex, higher-order cognition (Murray et al., 2017). Our approach adopts this extended view of corollary discharge. We utilize predicted post-action world state atoms, $\alpha_s.eff$—analogous to corollary discharge—to support a cognitive function: the evaluation and revision of goal monitors.

Similarly, the processes of pausing, updating, and unpausing goal monitors in our approach could correspond to the dynamic gating mechanisms of the cortico-basal ganglia-thalamocortical loop (Frank, 2005; O'Reilly, 2006). These gating mechanisms play an important role in both motor actions and cognitive operations, such as updating representations in working memory.

Dynamic gating involves the basal ganglia (BG) modulating prefrontal cortex (PFC) activity via control of thalamic inhibition. In the default state, neurons in the indirect (NoGo) pathway maintain tonic thalamic inhibition through the substantia nigra pars reticulata (SNr) or globus pallidus internal segment (GPi), preventing activation of the thalamocortical loop. Activation of the NoGo pathway amplifies thalamic inhibition, blocking the initiation of specific actions or updates.

In our method, this gating mechanism is analogous to the procedure for pausing the monitor and preparing updated conditions (Algorithm 1, lines 6 - 9). The monitor is first paused, then updated while in this inhibited state, and remains in memory in the paused state—awaiting reactivation. This bears a conceptual resemblance to how thalamic inhibition gates the release of prepared motor or cognitive representations in the cortico-basal ganglia-thalamocortical loop. The unpause procedure (Algorithm 2) parallels the function of the direct (Go) pathway. When striatal Go neurons fire, they inhibit the tonic activity of SNr/GPi neurons, disinhibiting the thalamus and activating the thalamocortical loop. This permits previously gated representations to propagate into working memory in the PFC, much like our monitors reactivate—regularly scanning the world state for the continued presence of their newly updated conditions.

These two neuroscientific analogs illustrate potential conceptual parallels between our method and some established principles of human cognition while extending their application to dynamic goal monitoring. By drawing on concepts related to predictive modeling and dynamic gating, our method provides a cognitively plausible framework for proactive and adaptive goal management. While our current implementation is symbolic, future work could explore more biologically grounded or neurocomputational extensions.

Appendix B: Evaluation-Based Revisions to Defeasible Goal Monitors

The method described in Section 2.1 represents an important advancement toward universal goal monitoring, offering a robust framework for proactive, anticipation-based management of goal monitors during intentional actions. However, it does not address scenarios involving agents with defeasible precondition beliefs (Section 1.2.3), which require a fundamentally different approach. While agents in the first scenario can proactively manage goal monitors by leveraging foreknowledge of the post-action world state, they lack comparable foreknowledge of changes to their knowledge state in this second case. As a result, agents must rely on detecting and responding to updates in their knowledge base. This appendix introduces several key observations about these scenarios to inform the development of a tailored dynamic goal monitoring approach.

One important observation is that scenarios involving defeasible precondition beliefs require a reactive, evaluation-based method for revising monitored conditions. Agents must be able to recognize when unexpected developments in the world—and corresponding updates to their knowledge state—necessitate revisions to their monitors. While this approach is reactive in the context of monitor revision, it retains a proactive dimension with respect to preventing goal failure, aligning with the broader goal-monitoring paradigm. Specifically, it seeks to update the definition of goal monitors before they trigger erroneously, provided the agent acquires updated knowledge before a triggering event occurs. In this sense, the approach is reactive only in its reliance on newly acquired knowledge.

A critical requirement of this approach is metacognitive capability. Agents must detect and resolve self-inconsistencies that arise as their knowledge evolves. Specifically, they must evaluate divergences between their current, updated beliefs—stored in their core knowledge base—and their prior assumptions, encoded within the monitored conditions of their goal monitors. This process of comparing, reconciling, and ensuring consistency between cognitive states, particularly memory components, is inherently metacognitive and relies on metamemory processes (Nelson, 1990; Johnson & Hirst, 1993). Effective metacognition enables agents to align their goal monitors with their latest understanding of the environment and task requirements.

Additionally, there are strong conceptual parallels between the challenges of this class of scenarios and those addressed by nonmonotonic and paraconsistent logic. Nonmonotonic logic offers techniques for revising defeasible inferences or beliefs based on new evidence, while paraconsistent logic addresses the coexistence and resolution of inconsistencies within a system. These parallels suggest that methods developed in these fields (e.g. Elgot-Drapkin & Perlis, 1990; Anderson et al., 2008; Josyula & M'Bale, 2013; Herron & Josyula, 2023) could inform strategies for managing defeasible and inconsistent beliefs in goal monitoring.

In summary, addressing scenarios involving defeasible beliefs about goal preconditions requires a reactive, metacognitive approach that reconciles inconsistencies across the agent's memory systems. Insights from prior research on nonmonotonic and paraconsistent logic may inform the development of methods to effectively manage the complexities of belief revision and inconsistency resolution in dynamic environments. Future work will develop, implement, and demonstrate such an approach, exploring its integration with existing frameworks and its broader implications for dynamic goal monitoring.

Appendix C: Computational-Overhead Evaluation

While the benefit of dynamic monitoring is evident in its ability to prevent premature goal abandonment, any such benefit must be weighed against the added computational cost. To assess this overhead, the MIDCA architecture was instrumented to log execution time for the relevant portions of the Act (Algorithm 1) and Interpret (Algorithm 2) phases. Fine-grained timestamps were captured using Python's time.perf_counter() function. To isolate the overhead introduced by dynamic monitoring (relative to the static monitoring baseline), timing logic was embedded directly within the routines responsible for evaluating and updating dynamic monitors. The total duration of these updates was logged across each run. Separately, the total system runtime was computed by summing the durations of all cognitive phases. This enabled calculation of the proportion of runtime attributable to dynamic monitoring, using the formula:

$$Dynamic\ Monitoring\ Overhead\% = \frac{Dynamic\ Monitoring\ Overhead\ Time}{Baseline\ Runtime\ (Static\ Monitoring)} \times 100$$

Dynamic Monitoring Overhead Time refers to the time spent evaluating and updating monitors in the Act and Interpret phases as a result of dynamic monitoring. Baseline Runtime (Static Monitoring) is computed as the observed total runtime minus the dynamic monitoring overhead time. This subtraction isolates the portion of runtime that would have been incurred under static monitoring alone, enabling calculation of the relative increase in processing time attributable specifically to dynamic monitoring.

Table 2 summarizes results from five representative runs using the baseline configuration, in which each goal was associated with two monitored conditions. In all cases, the overhead remained below 1% of total runtime. Notably, total runtime was dominated by the Plan phase in cycle 1, where MIDCA computes the initial plan—accounting for over 2,000 ms and more than 90% of runtime in most runs. When this initial planning time is excluded, the relative monitoring overhead increases but remains modest, ranging between 7% and 15%.

Having established that dynamic monitoring introduces minimal overhead under baseline conditions, performance under more complex monitoring configurations was subsequently evaluated. Specifically, scenarios in which each goal was associated with 5 or 50 monitored conditions were used to assess scalability. To simulate increased monitoring complexity without affecting behavior, inert monitored conditions—i.e., conditions present in the initial world state and guaranteed to remain true—were appended to each goal monitor. For example, atoms such as IN-CITY(loc01, city0) or IN-CITY(loc12, city1) were used as additional monitored conditions. These inert conditions increased the agent's monitoring workload while preserving actual behavior and outcomes, thereby isolating the computational cost of scaling. As in previous tests, timing logic was used to capture total dynamic monitoring time in both Act and Interpret phases. Results are summarized in Table 3. While Act-phase monitoring time remained relatively stable (typically between 8–10 ms), Interpret-phase monitoring time increased as the number of monitored conditions grew: from an average of 4.0 ms (2 mcs) to 6.3 ms (5 mcs) and 73.2 ms (50 mcs). This increase was driven almost entirely by the Interpret phase. This result aligns with expectations: Act-phase monitoring checks only monitors potentially affected by the current action's effects,

whereas the Interpret phase iterates through all monitors to determine whether any are paused and should be resumed.

Table 2. Dynamic Monitoring Overhead Across Five Runs (2 Monitored Conditions)

Run	Interpret Dynamic Monitor (ms)	Act Dynamic Monitor (ms)	Total Monitor Time (ms)	Total Runtime (ms)	Plan Phase, Cycle 1 (ms)	% Overhead	% Overhead (Excl. $Plan_1$)
1	5.50	8.52	14.02	2523.35	2379.54	0.56%	9.93%
2	3.80	10.39	14.19	2641.36	2491.95	0.54%	11.34%
3	3.90	9.00	12.91	2332.40	2184.12	0.56%	11.68%
4	2.81	7.23	10.04	2343.72	2199.83	0.43%	7.31%
5	3.97	10.97	14.94	2290.57	2130.19	0.66%	14.90%

Despite this growth, total monitoring overhead in the 50-mcs condition remained modest, averaging just 3.54%. Moreover, the scaling was found to be sublinear: while the number of monitored conditions increased 25X (from 2 to 50), the Interpret monitoring time increased by only a factor of ~18.3—suggesting that per-condition costs decrease at scale.

Table 3. Effect of Increasing Monitored Conditions on Monitoring Overhead

Number of Monitored Conditions in Configuration	Avg. Interpret Dynamic Monitor Time (ms)	Avg. Act Dynamic Monitor Time (ms)	Avg. Total Overhead (%)
2 mcs (baseline configuration)	3.99	~8.8	0.55%
5 mcs (incl. 3 “inert” mcs)	6.26	~8.8	0.59%
50 mcs (incl. 48 “inert” mcs)	73.19	~10.2	3.54%

These findings reinforce the scalability of the dynamic monitoring approach, particularly in domains where only a small subset of monitored conditions is typically paused at a given time. While the 50-mcs scenario was included primarily to evaluate worst-case computational scaling, such a configuration is unlikely to arise in practice. Most goals are expected to involve only a few mcs—often fewer than five—indicating that dynamic monitoring can be adopted in real-time systems without significant performance penalty.